

Speaker Notes

What to say for each slide of the presentation

Compensating Connectivity Restrictions in Quantum Annealers via Splitting and Linearisation Techniques

Seelbach Benkner, Lähner, Golyanik, Kliesch, Moeller arXiv:2507.12536v1 (2025)

CENG 591 Graduate Seminar
Denizcan Yilmaz 2444172

A speaking script for a ten-minute talk. Each section has the same title as the matching slide. The cue line tells you what to say first; the rest is the substance, written to sound natural when read aloud rather than recited from the slide. Acronyms are spelled out on first use. The figures on slides 6 and 11 are points to gesture at while explaining.

Estimated total speaking time: about 9 minutes 50 seconds at a comfortable conversational pace, leaving roughly 20 seconds of buffer for transitions and pauses inside the 10-minute limit.

Slide 1 Title Slide

20 seconds

Cue. Good morning everyone. Today I'll be presenting a paper on quantum annealing.

The paper is by Seelbach Benkner and his colleagues, posted on arXiv in July 2025. It tackles a hard physical limit in current quantum annealers, the connectivity restriction, by combining two classical optimisation techniques: splitting and linearisation.

Slide 2 Outline

20 seconds

Cue. Here's how the talk is structured.

I'll cover the problem and the formal QUBO setup, then the standard workaround called minor embedding and why it doesn't scale. The middle is the contribution itself: splitting, linearisation, damping. Then the proof, the experiments, and a few limitations.

Slide 3 Why This Paper Matters

55 seconds

Cue. Let's start with why this problem actually matters.

A lot of practical problems boil down to combinatorial optimisation: scheduling, vehicle routing, circuit layout, protein folding. They're NP-hard in general, and classical solvers struggle as the size grows.

Quantum annealers, mostly the D-Wave machines, were designed for exactly this kind of problem. They encode it as interacting quantum spins, and the system relaxes into its lowest-energy state, which is the optimum.

The catch is connectivity. On a real chip, each qubit can only interact with a small set of nearby qubits, fixed by the wiring. So if the problem needs an interaction between two distant qubits, you can't encode it directly. Five thousand qubits on the chip, but only about one hundred and fifty usable variables for a dense problem.

Slide 4 Problem Formulation

50 seconds

Cue. A quick bit of formal setup.

Everything in the paper is written as a QUBO, which stands for Quadratic Unconstrained Binary Optimisation. We have n variables that are zero or one, and we minimise a quadratic cost. The matrix Q holds the pairwise interactions, so Q_{ij} is just how much variables i and j interact.

The Ising form is mathematically equivalent, but uses spin variables in plus or minus one instead of zero or one. That matches the actual physical degrees of freedom on the chip. The conversion is just a linear shift, so we don't lose anything.

Most importantly, a non-zero entry in A requires a physical wire between two qubits on the chip. If that wire isn't there, the interaction simply can't be implemented. This single fact is the root of everything that follows.

Slide 5 The Standard Fix: Minor Embedding

50 seconds

Cue. So how does the community currently deal with this gap?

The standard answer is minor embedding. Since you can't wire two distant qubits directly, you bundle several physical qubits into what acts like one logical qubit. The bundle is called a chain, and a strong penalty term in the Hamiltonian forces the chain to agree.

It's painful for four reasons. Finding the embedding itself is NP-hard. You have to redo it for every new instance, which can take ten minutes or more. The chained qubits aren't free, so a five-thousand-qubit chip

gives only about one hundred and fifty usable logical qubits. And long chains hurt precision because the chain-strength penalty competes with the actual problem signal.

Slide 6 The Core Idea: Splitting the Coupling Matrix

60 seconds

Cue. *This is the main idea of the paper. Let's look at Figure 1 at the top.*

The first row shows a fully connected problem decomposed into the hardware graph, here a Chimera unit cell, plus a small leftover set of edges that the hardware can't wire. The second row shows the same thing after a random permutation: a different subset of edges now lands on hardware.

The idea is simple. Some interactions in A already have a physical wire, so we handle them exactly. The others get approximated by adding a single bias number to each qubit, and biases don't need any wires.

So we split A into A_Q , the part that fits the hardware, and A_L , everything else. The permutation matters because it changes which edges fall into which part each iteration.

Slide 7 Linearisation and Damping

50 seconds

Cue. *Now we need to actually deal with A_L , the part that doesn't fit the hardware.*

The trick is a first-order Taylor expansion. The quadratic function is replaced by its tangent at the current iterate. The result is linear in s , which means it implements as a per-qubit bias. No new wires needed.

The catch is that a Taylor approximation only stays accurate close to where you expanded. So we add a damping term that penalises jumping too far from the current iterate, with strength controlled by λ . On the discrete cube this damping also reduces to a per-qubit bias, and weak monotonicity is guaranteed once λ is at least the spectral norm of two times A_L .

Slide 8 The Full Iteration Step

45 seconds

Cue. *Putting the two pieces together, this is what the algorithm solves at every iteration.*

Three terms. The first is the exact quadratic part, A_Q , which goes straight to the annealer. The second is the linearised contribution from A_L : the vector $2A_L s^{(k)}$ is computed from the previous iterate, so when we plug it in we get a per-qubit bias. The third is the damping, also a per-qubit bias, pulling the new iterate toward the previous one.

From the hardware's view, this is one tractable subproblem. As a side note, the whole iteration is mathematically equivalent to proximal gradient descent.

Slide 9 Random Permutation & Algorithm

50 seconds

Cue. *One more important trick, and then I'll show the algorithm.*

If you always use the same split, the same edges always get the exact treatment, and error builds up in the same places. To avoid that, the authors apply a random permutation P_k at every iteration that reshuffles which problem variables map to which physical qubits. Averaged over time, every edge gets the exact treatment some of the time.

For λ , the trick is that a variable only flips when its entry in a force vector changes sign. So only the midpoints between sorted entries actually produce different outcomes; the default tests fifteen of those candidates per iteration and keeps the best.

The full algorithm is two nested loops: the outer loop draws a fresh permutation, the inner loop runs the λ sub-iterations.

Slide 10 Weak Monotonicity Proof

50 seconds

Cue. Let's look at why this works in theory. The proof is three short steps.

The theorem says: if λ is at least as large as the spectral norm of two times A_L , the energy never increases.

Step one: define a surrogate, which is the energy with A_L linearised plus the damping term. By the Taylor remainder theorem, for large enough λ , the surrogate is an upper bound on the true energy.

Step two: the algorithm picks the minimum of the surrogate.

Step three: chain those two facts, and the true energy at the new iterate is at most the true energy at the old. Convergence follows because there are only finitely many binary vectors.

Slide 11 Experimental Results

75 seconds

Cue. Now to the experimental results.

Setup, briefly. The experiments use the D-Wave Neal simulated-annealing solver restricted to the Pegasus hardware connectivity, with a smaller subset run on the real D-Wave Advantage. Two benchmarks: a regular spin-glass dataset of 270 instances, and 909 QMLib Max-Cut instances with 500 to 1,000 variables.

Key findings, four points. First, on small instances minor embedding does work, but its preprocessing alone takes up to ten minutes per instance while splitting and LNLS finish in under a second. Figure 6, panel (b), makes this concrete: minor embedding sits roughly two orders of magnitude above everything else in wall-clock time. Second, for instances above 150 variables minor embedding fails outright; splitting becomes competitive and matches LNLS at sub-problem size around thirty. Third, the damping ablation on 922 instances shows that when damping wins, it wins by a substantially larger margin than when it loses. Fourth, on the real D-Wave Advantage (Figure 10), the splitting curve tracks the LNLS curves for plus-and-minus-one edge weights; float-valued or wide-range integer weights fail because of limited chip precision.

Slide 12 Limitations & Conclusions

60 seconds

Cue. Let me close with an honest summary.

Limitations. The method only converges to a local minimum. Float and wide-range integer weights fail on D-Wave because of precision limits. There is no clean rule for when splitting beats LNLS. λ selection is heuristic.

Future work. Several directions are left open. First, validating the simulated-annealing extension sketched in Section 3.8 to escape local minima. Second, finding principled λ schedules instead of the heuristic sign-change candidates. Third, building predictors for when splitting outperforms LNLS, given a problem instance. Fourth, testing the method on newer hardware topologies such as Zephyr, and on gate-based annealers entirely.

Conclusions. This is the first time splitting and linearisation have been used for discrete QUBO on hardware-restricted annealers. The method avoids NP-hard preprocessing, comes with a clean monotonicity proof, and works on real D-Wave hardware. The bottom line is a clear trade-off: you give up an already non-existent global optimum, and in return you gain the ability to run problems that are simply too large to embed.

Slide 13 Thanks for Listening

5 seconds

Cue. Thank you for your attention. I'm happy to take questions.